INFORME TÉCNICO GLOBAL — 2ª ENTREGA TPI FOOD STORE (Unidades 4 a 8)

- **Asignatura:** Bases de Datos II · **Proyecto:** Food Store · **Motores:** PostgreSQL 18.4 (16+), MongoDB 9.0.4, SQLite (STRICT).
- **Repo:** `https://github.com/matiiiazc/db_food_store.git` (rama `main`; cada unidad versionada con su commit). Este informe resume lo actuado, cómo se probó, los resultados medidos (incluido el tiempo de restauración del simulacro de desastre), las decisiones de diseño y el uso de IA. Detalle completo: documentación por unidad referenciada en cada sección.
- **Regla transversal cumplida:** la base original `food_store_tp3` nunca fue destino de restore ni de operaciones destructivas; toda prueba de escritura se ejecutó sobre copias restauradas (`food_store_tp3_restore_*`, `food_store_tp3_restore_sim_*`, `food_store_tp3_u4_fnbc_*`) o dentro de transacciones con `ROLLBACK`; ninguna evidencia anterior fue sobrescrita (todo con marca de tiempo).

---

1. Unidad 4 — Normalización avanzada y redundancia (TP6)

Qué implementamos. - **Refactorización a FNBC** de `control_lote_almacen(lote_id, deposito_id, responsable_control_id)`: a partir de las DFs reales se identificó la violatoria `ResponsableControlID → DepositoID` (determinante no superclave; el esquema estaba en 3FN pero no en FNBC, con los tres atributos primos). Descomposición sin pérdida en `lote_responsable` (R1) y `responsable_deposito` (R2), unión garantizada por el criterio de superclave sobre el atributo común, y vista de compatibilidad `v_control_lote_almacen` que reconstruye la relación original por reunión natural. - **Migración transaccional reversible** (`04_migracion_transaccional_reversible.sql` + spec Kiro `TP6/specs/kiro_fnbc/`): toda la migración (DDL incluido — PostgreSQL ejecuta el DDL transaccionalmente) corre entre `BEGIN` y `COMMIT|ROLLBACK`, con verificación de equivalencia **dentro de la transacción** (bloque `DO` que cuenta ambos `EXCEPT`; `RAISE EXCEPTION` aborta si hay diferencias y el `COMMIT` — última sentencia — no se ejecuta). **Plan de reversión obligatorio** documentado en `design.md` §6 (renombre conservador o drop de los objetos derivados; el origen nunca se modifica, por lo que revertir no pierde datos). - **Desnormalización controlada** del panel "top 5 categorías del día": vista materializada `mv_top_categorias_dia` con índice único (habilita `REFRESH CONCURRENTLY`) y filtro de baja lógica embebido; auditoría de desincronización con `FULL JOIN + IS DISTINCT FROM`.

Cómo lo probamos. Sobre copia de trabajo con la instancia real `(501,30,801),(502,30,801),(503,31,802)`: equivalencia bidireccional `EXCEPT = 0` (ambos sentidos); corrida transaccional completa en una copia nueva restaurada de un dump verificado (`food_store_tp3_u4_fnbc_20261109_235630`, restore 30,27 s) con guarda anti-`food_store_tp3`/bases de sistema; `EXPLAIN ANALYZE` antes/después para la desnormalización; auditoría de desync.

Resultados (medidos). - Migración transaccional: **Corrida A (COMMIT verificado)** — `VERIFICACION OK: EXCEPT=0 en ambos sentidos (filas en vista=3)`, 3 filas en R1 / 2 en R2. **Corrida B (ROLLBACK demostrado)** — contaminación de una copia temporal con `(501,31,801)` (viola la DF: responsable 801 con dos depósitos) → `duplicate key ... responsable_deposito_pkey` a mitad de migración → `ROLLBACK` restaura el estado previo exacto (mismas 4 tablas/vista con 3/3/2/3 filas; instancia idéntica). Total: **0,68 s**. Evidencia: `TPI2_entrega2_cierre/evidencias/u4_migracion_transaccional_20261109_235808.txt` y `TP6/evidencia/`. - Desnormalización: **23,586 ms → 0,134 ms (~176×)**, buffers 5.241 → 9; auditoría de desincronización: **0 filas**.

Decisiones de diseño. Vista materializada en vez de columna precalculada por trigger (evita contención de escritura en `detalle_pedido`; el costo se traslada a un refresco amortizable; reversible con `DROP MATERIALIZED`

`VIEW` sin perder datos); el esquema FNBC hace **inrepresentable** la violación de la DF (la PK de R2 rechaza al responsable duplicado — demostrado en la Corrida B), a diferencia del esquema original que permitía la inconsistencia latente.

IA (OpenCode). Generación de scripts y specs Kiro a partir de la consigna; revisión crítica propia. Aceptado: estructura de la descomposición y el envoltorio transaccional. Descartado/corregido: nada en esta unidad (todas las propuestas se verificaron con corridas). DUIA/bitácora: `TP6/specs/kiro_fnbc/` + este informe.

---

2. Unidad 5 — Administración y seguridad

Qué implementamos. (A) Roles de mínimo privilegio: 4 roles de grupo (`rol_app_lectura/escritura/soporte/reportes`) + 4 de login (`app_web`, `soporte_app`, `reportes_app`, `admin_datos`), permisos a nivel de **columna** sobre `usuario` (soporte ve datos de cuenta sin `email`/`password_hash`), rotación de password solo vía `fn_resetear_password` (`SECURITY DEFINER`), membresías sin `ADMIN OPTION`, `REVOKE ... FROM PUBLIC` sobre objetos de seguridad. (B) Auditoría nativa activa (`log_connections`, `log_disconnections`, `log_statement=mod` vía `ALTER SYSTEM`). (C) Simulacro de acceso indebido: 7 logins fallidos + 1 exitoso vía `fn_autenticar`, lectura masiva de `usuario` (denegada por permisos), escalamiento `GRANT admin_datos` (denegado por falta de ADMIN OPTION); reconstrucción completa desde el log anonimizado; contención con rotación de credencial **sin restaurar backup**. (D) Anonimización determinista (`usuario_anon`) con equivalencia demostrada de agregaciones.

Cómo lo probamos. Verificación con `\du` + `information_schema.role_table_grants`/`role_column_grants` + `has_column_privilege`; el simulacro se ejecutó contra la propia base como rol de bajo privilegio (`app_web`) con cada paso registrado en `audit_login` y en el log del servidor; equivalencia de agregaciones con `EXCEPT=0` en ambos sentidos (11 filas idénticas).

Resultados (medidos). Denegaciones correctas en lectura masiva y escalamiento (`app_web_sigue_sin_ser_admin = true`); secuencia completa reconstruida desde el log; ninguna exfiltración efectiva.

Decisiones de diseño. Auditoría con herramientas nativas del motor (límites documentados frente a `pgaudit`); el log se trató como activo sensible y se anonimizó **antes** de entregarlo a la IA (protocolo DUIA). **Corrección de esta entrega (code review CR-03):** se detectó y corrigió que los `ALTER DEFAULT PRIVILEGES` otorgaban `DELETE` en tablas futuras a `rol_app_escritura` (`awd`), contradiciendo la política de baja lógica R7 — script corregido y `REVOKE` aplicado y verificado en la base (`aw`; evidencia `TPI2_entrega2_cierre/evidencias/cr03_default_privileges_20261110.txt`).

IA (OpenCode). Reconstrucción del incidente a partir del fragmento de log anonimizado: la IA afirmó exfiltración de hashes **sin sustento**; el equipo la **descartó con fundamento** (los permisos denegaban la lectura; se verificó contra `information_schema` y la secuencia real). Caso documentado como el más significativo de la bitácora (`U5_admin_seguridad/BITACORA.md`, `docs/reconstruccion_ia.md`).

---

3. Unidad 6 — Mantenimiento (TPI2_U6_mantenimiento)

Qué implementamos. Plan de mantenimiento completo y automatizado con OpenCode: 6 scripts PowerShell 5.1 + 2 SQL + documentación de procedimientos: `backup_foodstore.ps1` (`pg_dump -Fc` con TOC + SHA256 + meta, lectura), `verificar_backup.ps1` (integridad), `restore_foodstore.ps1` (base nueva con guardas anti-original/anti-sistema), `mantenimiento_rutinario.ps1` (integridad + `ANALYZE`; `VACUUM` opcional con autorización, `VACUUM FULL` bloqueado), `programar_backup.ps1` (Task Scheduler), `simulacro_desastre.ps1` (restore contra reloj + informe post-incidente).

Cómo lo probamos. Batería completa cronometrada (P0–PF) el 2026-11-09 sobre el clúster real (213 MB), con línea base del origen antes de empezar y verificación final de que no cambió (`ORIGEN_SIN_CAMBIOS=TRUE`); restore siempre a bases nuevas; casos de falla deliberados (guardas, dump corrupto, inexistente).

Resultados (medidos, ver `TPI2_U6_mantenimiento/evidencias/RESUMEN_CORRIDA_U6_20261109.md`).

Prueba	Resultado	Tiempo
Backup `pg_dump -Fc`	dump 14.485.062 B + TOC 114 líneas + SHA256	**11,61 s**
Verificación de integridad	3/3 (tamaño, SHA256, TOC legible)	0,72 s
Restore a base nueva	`pg_restore` exit 0	**20,06 s**
Comparación origen ↔ restaurada	**0 diferencias** (conteos: 601.205/200.401/50.000/20.000/4 + 13 tablas, 5 vistas, 2 MV, 44 funciones, 5 triggers, 29 índices)	3,65 s + 0,89 s
Mantenimiento rutinario	FK/CHECK validadas, huérfanos 0, dominios 0, `ANALYZE`	4,18 s
Respaldo programado (Task Scheduler)	disparó solo, `LastTaskResult=0`, backup real generado	corrida de tarea ~8,5 s
Simulacro de desastre (contra reloj)	restore exit 0 + verificaciones 0 diferencias + informe post-incidente	**total 22,93 s · restauración 21,62 s**
Negativos (4/4)	guardas bloquean original y `postgres`; dump corrupto → exit 1; inexistente → excepción	< 1 s c/u

Durante la corrida se detectaron y corrigieron 5 fallos reales de infraestructura/scripting (F1–F5: operador `??` de PS7 en PS 5.1, `NativeCommandError` con stderr+EAP Stop, colisión de función con el alias `r` que producía un **falso "SUPERADAS"**, `$PSScriptRoot` vacío en defaults bajo `powershell.exe -File` que rompía la tarea programada — confirmado con repro mínimo —, y un bug de un comando comparativo ad-hoc), todos re-probados. La tarea de prueba fue eliminada tras verificar; el respaldo diario productivo queda como comando documentado (`programar_backup.ps1 -At 02:30`).

Hallazgo de calidad de datos (causa raíz encontrada en el code review final, CR-01): 533.524 filas con `subtotal ≠ cantidad × precio_unitario` provienen de un bug del generador del seed del TP3 (aleatorio independiente); la restauración fue **fiel** (idénticas en origen y restaurada — verificado). Documentado como desvío de datos pre-existente, no de recuperación; fix del generador versionado en `CODE_REVIEW_FINAL_IA.md`.

IA (OpenCode). Generación de scripts/procedimientos + corrección de los fallos F1–F5 con repro mínimo. Decisiones: `VACUUM` no ejecutado (solo `ANALYZE`) por diseño; `.pgpass` eliminado tras la corrida; DUIA/bitácora: `TPI2_U6_mantenimiento/docs/DUIA_BITACORA_U6.md`.

---

4. Unidad 7 — NoSQL (U7_nosql_mongodb)

Qué implementamos. Modelado documental del subdominio pedidos en MongoDB (`food_store_nosql.pedidos`): documento `pedido` con `cliente` embebido (objeto) y `lineas` (arreglo) — patrón de agregado completo en una lectura, sin `$lookup`; precios/cantidades **históricos inmutables** copiados de `detalle_pedido` (no recalculados desde el catálogo); trazabilidad con `id_origen`. Migración de 5 pedidos reales de PostgreSQL + consultas (a)–(d) + equivalentes relacionales.

Cómo lo probamos. Ejecución real contra MongoDB 9.0.4 local (pymongo, equivalente a `mongosh --file`) con evidencia capturada; verificación relacional sobre `food_store_tp3` con semánticas equivalentes (`eliminado=false` ↔ `estado:"ACTIVO"`; join a `producto.nombre` ↔ match sobre `lineas.producto`).

Resultados (medidos). Coincidencia exacta en cantidad e IDs: (b) 5/5, (c) 1/1, (d) 3 pedidos > umbral 30.000 — sin diferencias entre documental y relacional sobre los mismos datos.

Decisión de diseño. PostgreSQL como fuente de verdad y MongoDB como vista documental complementaria de lectura (CQRS): la integridad referencial y de dominio (FK/CHECK/UNIQUE/ACID) del relacional lo hace más seguro para importes; el patrón de acceso "pedido completo" favorece al documental. Justificación completa con contraste de 2 afirmaciones de IA en `U7_nosql_mongodb/INFORME_U7.md`.

IA (OpenCode). Comparación PostgreSQL/MongoDB pedida a la IA y **contrastada contra el esquema real**: afirmación 1 (lectura de agregados sin JOIN) confirmada; afirmación 2 (integridad relacional superior) confirmada contra `schema.sql`. Ninguna corrección de modelo fue necesaria (verificado con la comparación).

---

5. Unidad 8 — Bases de datos móviles e integración (U8_tablet_cierre)

Qué implementamos. Base SQLite del dominio para tablet de feria (`schema_sqlite.sql`: 4 tablas `STRICT` con FK, importes en **centavos enteros** — sin punto flotante —, CHECKs de dominio; `PRAGMA foreign_keys=ON`), especificada en Kiro (`docs/kiro/{requirements,design,tasks}.md`, notación EARS) y generada con OpenCode; cola de sincronización `cola_sync` con semáforo (`semaforo.sql`: PENDIENTE → RECHAZADO/CONFLICTO/SIMULADO_APROBADO) que compara cada solicitud pendiente contra la instantánea del servidor (`snapshot_servidor.csv` desde PostgreSQL) **sin tocar PostgreSQL** (el móvil propone, el servidor dispone).

Cómo lo probamos. Casos C1 (catálogo local offline), C7a/C7b (CHECK cantidad 0 y FK producto inexistente — error de constraint, 0 filas nuevas), y sincronización con **conflictos probados**: C2 (precio modificado en servidor → CONFLICTO; luego aceptación → SIMULADO_APROBADO), C3 (stock insuficiente → RECHAZADO), C4 (producto dado de baja → RECHAZADO), C5 (sin diferencias → SIMULADO_APROBADO, PostgreSQL sin cambios verificado), C6 (idempotencia: PK duplicada por `solicitud_uuid` + no re-evaluación de resueltos). Borradores en transacciones con `COMMIT` / `ROLLBACK` demostrados.

Resultados (medidos). Todos los casos con evidencia capturada (`evidencia_pruebas_sqlite.txt`, `evidencia_casos.txt`, `evidencia_pg_antes/despues.txt`); PostgreSQL verificado sin cambios tras la simulación.

Decisiones de diseño. Centavos enteros en SQLite (descartada la propuesta de `precio REAL` por riesgo de punto flotante — la IA lo propuso y el equipo lo descartó con fundamento); recarga del caché con `PRAGMA foreign_keys=OFF` fuera de transacción para preservar referencias (el `DELETE` con FKs activas y el `ON CONFLICT DO UPDATE` directo fallaron y se descartaron con evidencia). Mapa de los 3 motores y plan de mantenimiento del TP8 materializado por la Unidad 6.

Code review final asistido por IA (esta entrega). Extensión del review del TP8 (3 hallazgos) a **todos los scripts de la cursada** (41): 8 hallazgos nuevos verificados contra la base, con decisión humana documentada por hallazgo, 1 corrección aplicada (CR-03, default privileges), fixes versionados para CR-01 (seed de subtotales) y CR-02 (trigger de stock solo en INSERT), y 24 scripts sin hallazgos. Documento completo: `TPI2_entrega2_cierre/docs/CODE_REVIEW_FINAL_IA.md`.

---

6. Capa de datos completa y transversales

- **Relacional** (PostgreSQL 18.4, `food_store_tp3`): 13 tablas, 5 vistas, 2 MV, 44 funciones, 5 triggers, 29 índices — respaldada/restaurada verificada en U6.
- **Documental** (MongoDB 9.0.4): `food_store_nosql.pedidos`.
- **Móvil** (SQLite STRICT): `foodstore_movil.db` + instantánea del servidor.
- **Versionado en Git:** todo en `main` de `db_food_store` (scripts, specs Kiro, evidencias, informes por unidad y este cierre).

- **Bitácoras de IA/DUIA:** por unidad (TP2, TP3, TP4, TP5, U5 `BITACORA.md`, U6 `DUIA_BITACORA_U6.md`, U8 `bitacora_ia.md` / `DUIA.md`) y consolidación global en `TPI2_entrega2_cierre/docs/DUIA_GLOBAL_ENTREGA2.md`.
- **Checklist de trazabilidad** objetivo oficial → evidencia: `TPI2_entrega2_cierre/docs/CHECKLIST_TRAZABILIDAD_ENTREGA2.md`.

7. Protocolo de uso de IA sobre datos (cumplimiento)

- Ningún script generado por IA se ejecutó sin lectura previa línea por línea; las ejecuciones se hicieron sobre copias, en transacciones reversibles o con respaldo previo (U6 lo evidencia end-to-end).
- No se enviaron datos personales/sensibles a la IA: el log del simulacro U5 se anonimizó antes; los datos de prueba son sintéticos; las credenciales de desarrollo no se repiten fuera de los scripts locales y el `.pgpass` usado en U6 se eliminó al terminar.
- Todo uso de IA está declarado (DUIA por unidad + global) con finalidad y prompts/specs empleados, y con la decisión aceptar/descartar justificada en cada caso.